

PANDAS

for Data Analytics

A 3-Hour Instructor-Led Workshop

Instructor Manual & Participant Handbook

From Zero to Real-World Data Analysis with Python Pandas

Series • DataFrames • Cleaning • GroupBy • Merge • Dates • EDA • Real Projects

Table of Contents

Workshop Overview	3
Workshop Agenda	4
Datasets Used in This Workshop	5
1. Introduction to Pandas	6
2. Pandas Data Structures	9
3. Reading and Writing Files.....	15
4. Exploring Data	17
5. Selecting Data	21
6. Data Cleaning	25
7. Data Manipulation.....	28
8. GroupBy and Aggregation	32
9. Merge, Join and Concatenate	35
10. Working with Date and Time	39
11. Performance and Optimization.....	42
12. Practical Hands-on Demo (40 Minutes)	44
13. Mini Exercises	49
14. Interview Questions (30) — Beginner to Advanced.....	54
15. Best Practices	58
16. Pandas Cheat Sheet	60
17. Summary.....	62
Pandas Learning Roadmap.....	63

Workshop Overview

Target Audience

- Students learning Python-based data analysis
- Software developers moving into data-centric roles
- Data analysts who want to formalize Pandas skills
- Beginners with basic Python knowledge (variables, loops, lists, dictionaries, functions)

Learning Objectives

By the end of this session, participants will be able to:

- Understand what Pandas is and why it is the backbone of Python data analysis
- Create and manipulate Series and DataFrames confidently
- Read and write data in CSV, Excel, JSON and HTML formats
- Explore, clean, and preprocess real-world, messy datasets
- Filter, sort, and index data using label- and position-based access
- Handle missing values using industry-appropriate strategies
- Perform aggregation and group-based analysis for business reporting
- Merge and join multiple datasets the way SQL users join tables
- Work confidently with date and time data
- Apply vectorized operations instead of slow Python loops
- Perform a basic exploratory data analysis (EDA) workflow end-to-end
- Apply Pandas to real-world analytics projects with confidence

Prerequisites

- Basic Python syntax (variables, lists, dictionaries, functions, loops)
- A working Python 3.9+ environment with pip
- A code editor or Jupyter Notebook

TRAINER NOTE

Do a 2-minute poll at the start: who has used Excel, who has used SQL, who has written a Python loop over a list of dictionaries. This tells you which analogies will land best throughout the session.

Workshop Agenda (Total: 3 Hours / 180 Minutes)

The timings below are approximate. Build in a 5-minute buffer roughly every hour for questions and technical hiccups.

#	Topic	Duration	Focus
1	Introduction	10 min	What is Pandas, why it matters, ecosystem
2	Pandas Basics & Installation	10 min	Install, import, NumPy relationship
3	Series & DataFrames	30 min	Creating and inspecting core data structures
4	Reading & Writing Files	15 min	CSV, Excel, JSON, HTML I/O
5	Exploring Data	10 min	head, info, describe, value_counts
6	Data Selection & Filtering	25 min	loc, iloc, boolean masks, query()
7	Data Cleaning	25 min	Missing values, duplicates, string cleaning
8	Data Manipulation	15 min	sort, apply, map, vectorization
9	GroupBy & Aggregation	20 min	Business-style grouped reporting
10	Merge, Join & Concatenate	15 min	Combining datasets like SQL joins
11	Date & Time	10 min	to_datetime, extraction, arithmetic
12	Practical Hands-on Demo	40 min	Student Performance Analytics System
13	Quiz, Interview Prep & Q&A	15 min	Rapid-fire review and open floor

Suggested Break Points

- Short 5-minute stretch break after Section 7 (Data Cleaning) — around the 90-minute mark
- Keep the Hands-on Demo (Section 12) uninterrupted — it is the anchor exercise of the workshop

TRAINER TIP

Run this as a live-coding session, not a slide read-through. Type the code in front of the class and deliberately make one small mistake per hour (e.g., a `KeyError`) to demonstrate how to read and fix errors — it builds confidence faster than a flawless demo.

Datasets Used in This Workshop

Every code example and every expected output shown in this manual was generated by running the code against the files listed below. Download all seven files (provided alongside this manual) into the same folder as your notebook or script before the workshop begins — every `read_csv()` / `read_excel()` / `read_json()` call in this manual assumes they're in the current working directory.

File	Size	Used For / Notes
students_raw.csv	41 rows, 8 cols	Primary teaching file. Deliberately messy: 3-8 missing marks, 1 duplicate row, inconsistent City casing. Used in Sections 3-5.
students_clean.csv	40 rows, 8 cols	The cleaned output of Section 6 — missing values filled, duplicate dropped, Name/City normalized. Used in Sections 7, 8, 10.
students_raw.xlsx / .json	same as above	Excel and JSON copies of students_raw.csv, for the <code>read_excel()</code> / <code>read_json()</code> demos in Section 3.
customers.csv	6 rows, 3 cols	Customer master list. Used with orders.csv in Section 9 (Merge, Join, Concatenate).
orders.csv	7 rows, 5 cols	Order transactions, including 2 orders with a CustID that doesn't exist in customers.csv — a deliberate mismatch to demonstrate all four join types.
sales.csv	180 rows, 7 cols	Store sales across 4 regions and 6 months. Used for the business-oriented GroupBy example (Section 8) and the entire Date & Time section (Section 10).

File	Size	Used For / Notes
student_performance_project.csv	61 rows, 7 cols	Dedicated capstone dataset for the Section 12 hands-on project — separate from students_raw/clean so the final project feels like a fresh, standalone challenge.

TRAINER TIP

Have every participant run `pd.read_csv('students_raw.csv').shape` at the very start of the session as a quick environment check — if it prints (41, 8), their setup is ready to go.

1. Introduction to Pandas

1.1 What is Pandas?

Pandas is an open-source Python library used for fast, flexible, and expressive data manipulation and analysis. Its name is derived from “Panel Data”, an econometrics term for multi-dimensional structured datasets.

At its core, Pandas gives Python two powerful data structures — the Series (1-dimensional) and the DataFrame (2-dimensional, table-like) — along with hundreds of functions to read, clean, transform, and analyze data.

1.2 Why Pandas?

- Handles real-world, messy, tabular data with ease (missing values, mixed types, duplicates)
- Reads and writes almost any file format: CSV, Excel, JSON, SQL, HTML, Parquet
- Built on top of NumPy, so it is fast and vectorized
- Integrates seamlessly with Matplotlib, Seaborn, Scikit-learn, and SQL databases
- Massive community and industry adoption — the de facto standard for data analysis in Python

1.3 A Brief History of Pandas

Pandas was created by Wes McKinney in 2008 while working at AQR Capital Management, a hedge fund, where he needed a tool for high-performance, flexible analysis of financial data. It was open-sourced in 2009 and has since become one of the most downloaded Python libraries in the world, forming the foundation of the modern PyData ecosystem alongside NumPy, Matplotlib, and Jupyter.

1.4 Why Data Analysts Love Pandas

- One line of code can replace dozens of lines of manual loop-based logic
- Readable, expressive syntax close to how analysts think (“group by city, average the sales”)
- Works equally well for small CSV files and multi-million row datasets
- Excellent for the entire analytics pipeline: import → clean → analyze → export → visualize

1.5 Pandas vs. Python Lists and Dictionaries

Feature	Lists / Dictionaries	Pandas
Labeled rows & columns	No	Yes (Index & Columns)
Vectorized math operations	No (needs loops)	Yes
Handling missing data	Manual	Built-in (NaN, isnull, fillna)
File I/O (CSV/Excel/JSON)	Manual parsing	One line of code
Performance on large data	Slow	Fast (backed by NumPy/C)
Group/aggregate operations	Manual loops	groupby() in one line

1.6 Pandas vs. Excel

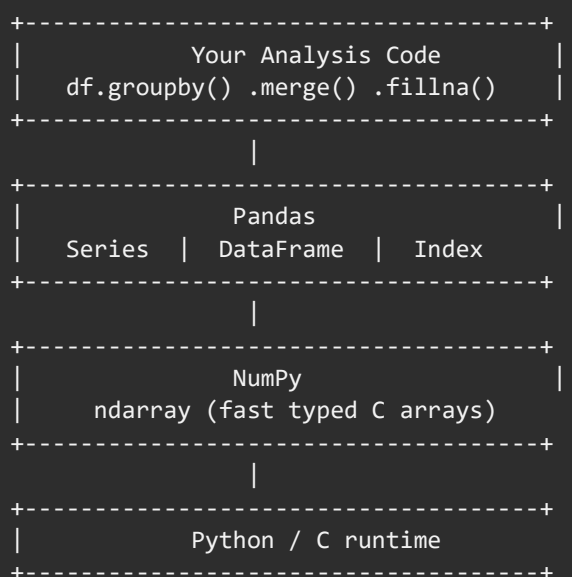
Feature	Excel	Pandas
Row limit	~1,048,576 rows	Limited only by RAM (millions+)
Automation & repeatability	Manual, error-prone	Fully scriptable, repeatable
Version control	Difficult (binary files)	Plain-text code, Git-friendly
Complex joins/transforms	VLOOKUP, Power Query	merge(), join(), one-liners
Integration with ML/BI tools	Limited	Native (Scikit-learn, SQL, APIs)

REAL-WORLD EXAMPLE

A retail analyst receiving a 500,000-row daily sales CSV cannot comfortably open it in Excel. In Pandas: `pd.read_csv('sales.csv')` loads it in under a second, and `df.groupby('store')['revenue'].sum()` gives store-wise totals instantly.

1.7 Pandas Architecture

Pandas is built directly on top of NumPy arrays, which provide fast, contiguous, typed memory storage. Pandas adds labeled axes (index and columns), heterogeneous column types, and a rich API on top of that foundation.



1.8 Relationship Between NumPy and Pandas

- Every Pandas Series is, internally, backed by a NumPy array (or an extension array)

- Pandas adds an Index (row labels) and Columns (for DataFrames) on top of NumPy's raw arrays
- Mathematical and statistical functions in Pandas (mean, std, sum) are powered by NumPy under the hood
- You can freely convert between them: `df.values / df.to_numpy()` and `pd.DataFrame(numpy_array)`

1.9 Installation

```
# Using pip
pip install pandas

# Using conda
conda install pandas

# Verify installation
python -c "import pandas as pd; print(pd.__version__)"
```

Output:

2.2.2

1.10 Importing Pandas

The universal convention, used in virtually every codebase and tutorial, is to import Pandas with the alias `pd`:

```
import pandas as pd
import numpy as np # commonly imported alongside Pandas
```

CONVENTION

Always use 'import pandas as pd'. Every function, tutorial, and Stack Overflow answer assumes this alias — deviating from it only confuses readers of your code.

✓ KEY TAKEAWAYS

- Pandas provides two core structures — Series and DataFrame — for labeled, tabular data analysis.
- It is built on NumPy, giving it speed, but adds labels and heterogeneous data handling NumPy lacks.
- Pandas beats raw Python lists/dicts and Excel for anything beyond small, one-off, manual data tasks.

✗ COMMON MISTAKES

- Forgetting to alias the import as `pd`, leading to verbose, non-standard code.
- Assuming Pandas replaces NumPy entirely — in practice they are complementary, not competitors.
- Trying to learn every function on day one instead of the ~20 that cover 90% of daily work.

★ TRAINER TIP

- Open with a live side-by-side: the same 'average sales by city' task done with a raw Python loop versus one line of Pandas `groupby()`. The contrast sells the entire workshop in 60 seconds.

? STUDENT Q&A

Q: Is Pandas only for small datasets?

A: No — it comfortably handles millions of rows in memory; for data larger than RAM, tools like Dask or Polars extend the same API.

Q: Do I need to know NumPy before learning Pandas?

A: Not strictly, but basic familiarity with arrays and vectorized thinking makes Pandas click faster.

Q: Is Pandas free for commercial use?

A: Yes, it is released under the permissive BSD 3-Clause license.

⚙ **INTERVIEW INSIGHT**

Interviewers often open with 'What is Pandas and why would you choose it over Excel or raw Python?' — a confident answer covering vectorization, labeled data, and file I/O signals real hands-on experience.

© Shalinee Priya

2. Pandas Data Structures

2.1 The Series — a 1-Dimensional Labeled Array

A Series is a one-dimensional array-like object with an attached index (row labels). Think of it as a single labeled column.

Creating a Series from a List

```
import pandas as pd

marks = pd.Series([85, 90, 78, 92])
print(marks)
```

Output:

```
0    85
1    90
2    78
3    92
dtype: int64
```

Creating a Series from a Dictionary

```
city_population = pd.Series({'Delhi': 33800000, 'Mumbai': 20700000, 'Pune': 6800000})
print(city_population)
```

Output:

```
Delhi      33800000
Mumbai     20700000
Pune       6800000
dtype: int64
```

Creating a Series from a NumPy Array

```
import numpy as np
arr = np.array([10, 20, 30])
s = pd.Series(arr)
print(s)
```

Output:

```
0    10
1    20
2    30
dtype: int64
```

Custom Index

```
marks = pd.Series([85, 90, 78], index=['Math', 'Science', 'English'])
print(marks)
```

Output:

```
Math      85
Science   90
```

```
English    78
dtype: int64
```

Series Data Types (dtype)

A Series stores homogeneous data — every element shares a dtype such as int64, float64, object (text), bool, or datetime64. Pandas infers this automatically, but you can force a type with `astype()`.

Useful Series Attributes — With Demo Code

Every example below uses the same Series so you can run them in order on your own machine:

```
marks = pd.Series([85, 90, 78],
index=['Math', 'Science', 'English'],
name='Marks')
```

Attribute	Description
<code>s.values</code>	Underlying NumPy array of values
<code>s.index</code>	The row labels
<code>s.dtype</code>	Data type of the elements
<code>s.shape</code>	Tuple with the number of elements
<code>s.size</code>	Total number of elements
<code>s.name</code>	Optional name of the Series

```
print(marks.values)
print(marks.index)
print(marks.dtype)
print(marks.shape)
print(marks.size)
print(marks.name)
```

Output:

```
[85 90 78]
Index(['Math', 'Science', 'English'],
dtype='object')
int64
(3,)
3
Marks
```

Useful Series Methods — With Demo Code

Method	Description
<code>s.head(n)</code> / <code>s.tail(n)</code>	First / last n elements
<code>s.sum()</code> / <code>s.mean()</code> / <code>s.max()</code> / <code>s.min()</code>	Aggregate statistics
<code>s.sort_values()</code>	Sort by value
<code>s.unique()</code> / <code>s.nunique()</code>	Distinct values / count of distinct values
<code>s.apply(func)</code>	Apply a function element-wise

Method	Description
<pre>print(marks.head(2)) # first 2 elements print(marks.tail(2)) # last 2 elements print(marks.sum(), marks.mean(), marks.max(), marks.min()) print(marks.sort_values()) print(marks.unique()) # array of distinct values print(marks.nunique()) # count of distinct values print(marks.apply(lambda x: x + 5)) # add 5 to every element</pre>	
Output: <pre>Math 85 Science 90 Name: Marks, dtype: int64 Science 90 English 78 Name: Marks, dtype: int64 253 84.33333333333333 90 78 English 78 Math 85 Science 90 Name: Marks, dtype: int64 [85 90 78] 3 Math 90 Science 95 English 83 Name: Marks, dtype: int64</pre>	

2.2 The DataFrame — a 2-Dimensional Labeled Table

A DataFrame is a 2-dimensional, size-mutable, tabular data structure with labeled rows (index) and columns — conceptually similar to an Excel sheet or a SQL table. In fact, a DataFrame can be thought of as a dictionary of Series that all share the same index.

Creating a DataFrame from a Dictionary of Lists

```
data = {
    'Name': ['Aditi', 'Rohan', 'Meera'],
    'Age': [21, 23, 22],
    'City': ['Delhi', 'Mumbai', 'Pune']
}
df = pd.DataFrame(data)
print(df)
```

Output:

```
   Name  Age  City
0  Aditi   21  Delhi
1  Rohan   23  Mumbai
```

2	Meera	22	Pune
---	-------	----	------

Creating a DataFrame from a List of Dictionaries

```
records = [
    {'Name': 'Aditi', 'Age': 21},
    {'Name': 'Rohan', 'Age': 23},
]
df = pd.DataFrame(records)
print(df)
```

Output:

	Name	Age
0	Aditi	21
1	Rohan	23

Creating a DataFrame from Nested Lists

```
rows = [['Aditi', 21], ['Rohan', 23]]
df = pd.DataFrame(rows, columns=['Name', 'Age'])
print(df)
```

Output:

	Name	Age
0	Aditi	21
1	Rohan	23

Creating a DataFrame from a NumPy Array

```
arr = np.array([[1, 2], [3, 4]])
df = pd.DataFrame(arr, columns=['A', 'B'])
print(df)
```

Output:

	A	B
0	1	2
1	3	4

Creating a DataFrame from CSV, Excel, and JSON

```
df_csv = pd.read_csv('students_raw.csv')
df_excel = pd.read_excel('students_raw.xlsx', sheet_name='Students')
df_json = pd.read_json('students_raw.json')
```

NOTE

`read_csv`, `read_excel` and `read_json` are covered in full detail in Section 3 — Reading and Writing Files.

2.3 Understanding a DataFrame's Anatomy

We'll use this DataFrame for every demo in this section:

```
df = pd.DataFrame({'Name':
    ['Aditi', 'Rohan', 'Meera'],
```

```
'Age': [21, 23, 22],
'City':
['Delhi', 'Mumbai', 'Pune']})
```

Concept	Meaning	Access
Rows	Individual records / observations	df.loc[], df.iloc[]
Columns	Fields / variables/ features	df['col'], df.columns
Index	Row labels (default: 0,1,2,...)	df.index
Shape	(rows, columns) as a tuple	df.shape
Size	Total number of cells (rows × cols)	df.size
Data Types	dtype of every column	df.dtypes
Memory Usage	Bytes consumed per column	df.memory_usage()

Demo: Rows

```
print(df.loc[0])          # row with label 0
print(df.iloc[-1])       # last row by position
```

Output:

```
Name    Aditi
Age      21
City    Delhi
Name: 0, dtype: object
Name    Meera
Age      22
City    Pune
Name: 2, dtype: object
```

Demo: Columns

```
print(df['Name'])         # single column
print(df.columns)        # all column names
```

Output:

```
0    Aditi
1    Rohan
2    Meera
Name: Name, dtype: object
Index(['Name', 'Age', 'City'], dtype='object')
```

Demo: Index

```
print(df.index)
```

Output:

```
RangeIndex(start=0, stop=3, step=1)
```

Demo: Shape, Size, Data Types and Memory Usage

```
print(df.shape)          # (3, 3)
```

```
print(df.size)           # 9
print(df.dtypes)
print(df.memory_usage(deep=True))
```

Output:

```
(3, 3)
9
Name    object
Age     int64
City    object
dtype: object
Index    132
Name     186
Age       24
City     174
dtype: int64
```

✓ KEY TAKEAWAYS

- A Series is a labeled 1-D array; a DataFrame is a labeled 2-D table — essentially a dict of aligned Series.
- DataFrames can be built from dicts, lists, NumPy arrays, or directly loaded from files.
- `shape`, `size`, `dtypes`, and `memory_usage()` are the first commands to run on any new DataFrame.

✗ COMMON MISTAKES

- Confusing `shape` (rows, columns) with `size` (total number of cells).
- Mixing data types within a column unintentionally, causing `dtype` to fall back to 'object'.
- Forgetting that a dictionary of lists requires all lists to be the same length.

★ TRAINER TIP

- Draw the DataFrame as a literal table on the whiteboard with the Index column highlighted in a different color — students consistently forget the index is not a regular column.

? STUDENT Q&A**Q: What is the difference between a DataFrame and a Python dictionary of lists?**

A: A DataFrame adds an aligned index, vectorized operations, and a huge built-in API — a raw dict of lists has none of that.

Q: Can a DataFrame have columns of different data types?

A: Yes — that is the whole point. Each column is its own Series with its own dtype, unlike a NumPy 2D array which must be homogeneous.

Q: What does dtype 'object' actually mean?

A: It usually means the column holds strings (or a genuinely mixed type) — Pandas stores these as Python objects rather than a fixed-width type.

🔧 INTERVIEW INSIGHT

A very common interview question is: 'How is a DataFrame related to a Series?' The strong answer: a DataFrame is essentially a dictionary of Series objects that all share a common index.

3. Reading and Writing Files

Real-world data rarely arrives as a hand-typed dictionary — it arrives as a CSV export, an Excel workbook, a JSON API response, or a table on a webpage. Pandas provides a consistent `read_*` / `to_*` family of functions to handle all of these.

WORKSHOP DATASET

From this point on, every example in this manual uses the real files provided with this workshop: `students_raw.csv` (40 messy student records), `students_clean.csv` (the cleaned version produced in Section 6), `customers.csv` / `orders.csv` (for merges), and `sales.csv` (180 rows of store sales, for `GroupBy` and `Date/Time`). Keep them in the same folder as your notebook/script. See the full dataset list in the Workshop Overview section.

3.1 Reading Files

`read_csv()`

```
df = pd.read_csv('students_raw.csv')
df = pd.read_csv('students_raw.csv', usecols=['Name', 'City', 'Math'])
df = pd.read_csv('students_raw.csv', parse_dates=['JoinDate'])
print(df.head())
```

Output:

	StudentID	Name	Gender	City	JoinDate	Math	Science	English
0	1	Aditi	M	Delhi	2024-01-10	95.0	63.0	98.0
1	2	Rohan	F	Chennai	2024-01-19	62.0	77.0	77.0
2	3	Meera	F	Chennai	2024-01-28	45.0	76.0	61.0
3	4	Karan	M	Mumbai	2024-02-06	NaN	65.0	61.0
4	5	Sneha	M	Delhi	2024-02-15	75.0	89.0	61.0

`read_excel()`

```
df = pd.read_excel('students_raw.xlsx', sheet_name='Students')
# requires: pip install openpyxl
```

`read_json()`

```
df = pd.read_json('students_raw.json')
```

`read_html()`

```
tables = pd.read_html('https://example.com/stats-page')
df = tables[0] # returns a LIST of DataFrames, one per <table>
# requires: pip install lxml
```

3.2 Writing Files

```
df.to_csv('students_output.csv', index=False)
df.to_excel('students_output.xlsx', sheet_name='Report', index=False)
df.to_json('students_output.json', orient='records', indent=2)
```

IMPORTANT

Always pass `index=False` when writing to CSV/Excel unless you specifically want the DataFrame's index saved as a column — forgetting this is the single most common cause of a stray 'Unnamed: 0' column on the next read.

3.3 Common Errors While Importing Datasets

Error	Typical Cause	Fix
<code>FileNotFoundError</code>	Wrong path or working directory	Use an absolute path or check <code>os.getcwd()</code>
<code>UnicodeDecodeError</code>	Non-UTF-8 file encoding	<code>pd.read_csv(file, encoding='latin1')</code>
<code>ParserError: tokenizing data</code>	Inconsistent delimiters / extra commas	Specify <code>sep=' '</code> or <code>on_bad_lines='skip'</code>
Unnamed: 0 column appears	Index was saved on export	Use <code>index=False</code> on <code>to_csv</code> / <code>to_excel</code>
Dates read as text (object)	Dates not parsed automatically	Use <code>parse_dates=['col']</code> in <code>read_csv</code>

✓ KEY TAKEAWAYS

- `read_csv`, `read_excel`, `read_json`, and `read_html` cover the vast majority of real-world data ingestion.
- The matching `to_*`() functions write data back out in the same formats.
- Always set `index=False` on export unless you deliberately want the index preserved.

✗ COMMON MISTAKES

- Not specifying encoding for files exported from Excel or Windows systems (commonly needs 'latin1' or 'cp1252').
- Forgetting `parse_dates`, leaving date columns as plain text objects.
- Assuming `read_html` returns a single DataFrame — it always returns a list.

★ TRAINER TIP

- Bring one deliberately messy CSV (extra header row, mixed delimiters) and let students hit the `ParserError` live — then fix it together. Debugging real errors sticks far better than a clean happy-path demo.

? STUDENT Q&A

Q: Can Pandas read directly from a URL?

A: Yes — `read_csv`, `read_excel`, and `read_html` can all accept a URL string directly in place of a file path.

Error	Typical Cause	Fix
Q: What if my Excel file has multiple sheets? A: Pass <code>sheet_name=None</code> to <code>read_excel</code> to get back a dictionary of DataFrames, one per sheet. Q: Is there a faster alternative to CSV for large files? A: Yes — Parquet (<code>to_parquet</code> / <code>read_parquet</code>) is columnar, compressed, and much faster for large datasets. 🔧 INTERVIEW INSIGHT <i>Interviewers like to ask how you would handle a CSV that fails to load. A strong answer walks through encoding issues, delimiter mismatches, and bad line handling — showing real debugging experience, not just textbook syntax.</i>		

4. Exploring Data

Before cleaning or analyzing anything, every experienced analyst runs a standard first-look routine on a new DataFrame. This section covers that routine.

4.1 The First-Look Toolkit

Function / Attribute	What It Shows	When to Use
<code>df.head(n)</code>	First n rows (default 5)	Quick sanity check after loading
<code>df.tail(n)</code>	Last n rows (default 5)	Check how the file ends (footers, totals)
<code>df.sample(n)</code>	n random rows	Spot-check data quality across the file
<code>df.info()</code>	Column dtypes, non-null counts, memory	First command after every <code>read_csv()</code>
<code>df.describe()</code>	Summary statistics for numeric columns	Quick distribution overview
<code>df.shape</code>	(rows, columns)	Confirm the full file loaded
<code>df.columns</code>	List of column names	Check exact spelling/casing
<code>df.index</code>	Row label range	Confirm default vs custom index
<code>df.dtypes</code>	Data type per column	Spot columns that loaded as the wrong type
<code>df['col'].value_counts()</code>	Frequency of each unique value	Understand categorical columns
<code>df['col'].unique()</code>	Array of distinct values	See exactly what categories exist
<code>df['col'].nunique()</code>	Count of distinct values	Quick cardinality check

Demo Code for Every Function in the Table

We'll run every function in the table above against the real `students_raw.csv` file (40 students, with a few missing marks, a duplicate row, and some inconsistent City casing — deliberately messy, exactly like a real export):

```
df = pd.read_csv('students_raw.csv')
```

```
print(df.head(2))
```

Output:

```
   StudentID  Name Gender  City  JoinDate  Math  Science  English
0          1  Aditi    M  Delhi  2024-01-10  95.0    63.0    98.0
1          2  Rohan    F  Chennai  2024-01-19  62.0    77.0    77.0
```

```
print(df.tail(2))
```

Output:

```
   StudentID  Name Gender  City  JoinDate  Math  Science  English
39         40    Om     F  Chennai  2024-12-26  42.0    85.0    96.0
40          1  Aditi    M  Delhi  2024-01-10  95.0    63.0    98.0  <- the duplicate row
```

```
print(df.sample(2, random_state=1))
```

Output:

	StudentID	Name	Gender	City	JoinDate	Math	Science	English
3	4	Karan	M	Mumbai	2024-02-06	NaN	65.0	61.0
2	3	Meera	F	Chennai	2024-01-28	45.0	76.0	61.0

```
print(df.info())
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 41 entries, 0 to 40
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
0   StudentID   41 non-null    int64
1   Name        41 non-null    object
2   Gender      41 non-null    object
3   City        41 non-null    object
4   JoinDate    41 non-null    object
5   Math        38 non-null    float64
6   Science     39 non-null    float64
7   English     38 non-null    float64
dtypes: float64(3), int64(1), object(4)
memory usage: 2.7+ KB
```

READ THE OUTPUT

info() already tells a story before you've written a single cleaning line: Math has 41-38=3 missing values, Science has 2, and English has 3 — and JoinDate is still plain text (object), not a real date. Both are addressed in Sections 6 and 10.

```
print(df.describe())
```

Output:

	StudentID	Math	Science	English
count	41.000000	38.000000	39.000000	38.000000
mean	20.024390	67.315789	66.128205	65.815789
std	11.938358	18.528437	17.508163	18.800309
min	1.000000	35.000000	36.000000	36.000000
25%	10.000000	50.500000	53.000000	51.000000
50%	20.000000	68.500000	67.000000	63.500000
75%	30.000000	84.750000	77.000000	79.000000
max	40.000000	95.000000	98.000000	98.000000

```
print(df.shape)
```

Output:

```
(41, 8)
```

```
print(df.columns)
```

Output:

```
Index(['StudentID', 'Name', 'Gender', 'City', 'JoinDate', 'Math', 'Science',
      'English'],
      dtype='object')
```

```
print(df.index)
```

Output:

```
RangeIndex(start=0, stop=41, step=1)
```

```
print(df.dtypes)
```

Output:

```
StudentID      int64
Name           object
Gender         object
City           object
JoinDate       object
Math           float64
Science        float64
English        float64
dtype: object
```

```
print(df['City'].value_counts())
```

Output:

```
City
Chennai    11
Pune        9
Delhi       8
Mumbai      6
Kolkata     5
MUMBAI      1
delhi       1
Name: count, dtype: int64
```

```
print(df['City'].unique())
```

Output:

```
['Delhi' 'Chennai' 'Mumbai' 'Kolkata' 'Pune' 'MUMBAI' 'delhi']
```

```
print(df['City'].nunique())
```

Output:

```
7
```

WHY THIS MATTERS

nunique() just reported 7 cities when there are really only 5 — 'MUMBAI' and 'delhi' are being counted as separate categories purely because of casing. This is the exact kind of issue Section 6 (Data Cleaning) fixes with .str.title().

Example: A Complete First Look

```
df = pd.read_csv('students_raw.csv')
print(df.head())
print(df.info())
print(df.describe())
print(df['Gender'].value_counts())
```

Output:

	StudentID	Name	Gender	City	JoinDate	Math	Science	English
0	1	Aditi	M	Delhi	2024-01-10	95.0	63.0	98.0
1	2	Rohan	F	Chennai	2024-01-19	62.0	77.0	77.0

```

...
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 41 entries, 0 to 40
Data columns (total 8 columns): ... (see info() output above)
   StudentID   Math   Science   English
count  41.000000  38.000000  39.000000  38.000000
mean    20.024390  67.315789  66.128205  65.815789
...
Gender
M      21
F      20
Name: count, dtype: int64

```

✓ KEY TAKEAWAYS

- head/tail/sample give a quick visual check; info() and describe() give the real structural picture.
- info() is the single most useful first command — it reveals dtypes and missing values in one call.
- value_counts(), unique(), and nunique() are essential for understanding categorical columns.

✗ COMMON MISTAKES

- Only ever calling head() and never info() — missing values and wrong dtypes go unnoticed.
- Assuming describe() covers text columns by default (it only covers numeric columns unless include='all' is passed).
- Confusing unique() (the values) with nunique() (the count of values).

★ TRAINER TIP

- Teach this as a fixed 4-step ritual for every new dataset: shape → info() → describe() → head(). Students who internalize a routine explore far more confidently than those improvising each time.

? STUDENT Q&A

Q: What's the difference between describe() and info()?

A: info() shows structure — dtypes and non-null counts; describe() shows statistics — mean, std, min, max, quartiles.

Q: How do I see summary stats for text columns too?

A: Pass df.describe(include='all') or df.describe(include='object').

Q: Why does sample() sometimes give different results each run?

A: It is random by default; pass random_state=42 to make it reproducible.

⚙️ INTERVIEW INSIGHT

A common scenario question: 'You just loaded a new CSV — walk me through your first five commands.' Interviewers are testing for a disciplined exploration habit, not memorized syntax.

5. Selecting Data

This section (and Section 6 onward) uses `students_raw.csv` — the same 41-row file introduced in Section 4. Load it once at the top of your notebook:

```
df = pd.read_csv('students_raw.csv')
```

5.1 Selecting Columns

```
print(df['Name'].head(3))          # single column -> Series
print(df[['Name', 'City']].head(3)) # multiple columns -> DataFrame
```

Output:

```
0    Aditi
1    Rohan
2    Meera
Name: Name, dtype: object
```

```
      Name    City
0  Aditi  Delhi
1  Rohan  Chennai
2  Meera  Chennai
```

5.2 loc[] vs iloc[]

	loc[]	iloc[]
Basis	Label-based	Position-based (integer)
End of slice	Inclusive	Exclusive (like Python lists)
Example	<code>df.loc[0:2, 'Name']</code>	<code>df.iloc[0:2, 0]</code>

```
print(df.loc[0])
# row with label 0
print(df.loc[0:2, ['Name', 'City']])
# rows 0-2 (inclusive), selected columns
print(df.iloc[0:2, 0:2])
# first 2 rows, first 2 columns
```

Output:

```
StudentID      1
Name          Aditi
Gender         M
City          Delhi
JoinDate      2024-01-10
Math          95.0
Science        63.0
English        98.0
Name: 0, dtype: object
```

```
      Name    City
0  Aditi  Delhi
1  Rohan  Chennai
```


		loc[]	iloc[]
2	Meera	Chennai	
	StudentID	Name	
0	1	Aditi	
1	2	Rohan	

5.3 Slicing

```
print(df[1:4][['StudentID', 'Name', 'City']]) # rows at position 1,2,3
```

Output:

	StudentID	Name	City
1	2	Rohan	Chennai
2	3	Meera	Chennai
3	4	Karan	Mumbai

5.4 Boolean Indexing & Conditional Filtering

```
print(df[df['Math'] > 85][['StudentID', 'Name', 'City', 'Math']])
```

```
print(df[(df['Math'] > 70) & (df['City'] == 'Pune')][['StudentID', 'Name', 'City', 'Math']])
# AND -> &
```

Output:

	StudentID	Name	City	Math
0	1	Aditi	Delhi	95.0
9	10	Rahul	Chennai	88.0
11	12	Kabir	Delhi	86.0
12	13	Ishita	Mumbai	88.0
14	15	neha	Pune	92.0
20	21	Ritu	Kolkata	88.0
22	23	Sanya	MUMBAI	86.0
40	1	Aditi	Delhi	95.0

	StudentID	Name	City	Math
14	15	neha	Pune	92.0
24	25	Kavya	Pune	85.0
35	36	Kunal	Pune	71.0

IMPORTANT

Always use & / | with parentheses around each condition — Python's 'and' / 'or' keywords do NOT work element-wise on a Series and will raise a ValueError.

5.5 isin()

```
print(df[df['City'].isin(['Delhi', 'Pune'])][['StudentID', 'Name', 'City']].head(6))
```

Output:

	StudentID	Name	City
0	1	Aditi	Delhi

4	5	Sneha	Delhi
6	7	Priya	Pune
11	12	Kabir	Delhi
13	14	Farhan	Pune
14	15	neha	Pune

5.6 between()

```
print(df[df['Math'].between(80, 90)][['StudentID', 'Name', 'Math']]) # inclusive on both ends
```

Output:

	StudentID	Name	Math
8	9	Anjali	84.0
9	10	Rahul	88.0
11	12	Kabir	86.0
12	13	Ishita	88.0
20	21	Ritu	88.0
22	23	Sanya	86.0
24	25	Kavya	85.0
25	26	Nikhil	85.0
26	27	Simran	84.0
31	32	Rajat	84.0

5.7 query()

`query()` lets you filter using a readable string expression instead of bracket-heavy boolean masks — many analysts find it closer to SQL's WHERE clause.

```
print(df.query("Math > 70 and City == 'Pune'")[['StudentID', 'Name', 'City', 'Math']])
```

Output:

	StudentID	Name	City	Math
14	15	neha	Pune	92.0
24	25	Kavya	Pune	85.0
35	36	Kunal	Pune	71.0

✓ KEY TAKEAWAYS

- `loc` is label-based and inclusive of the end of a slice; `iloc` is position-based and exclusive.
- Boolean masks combined with `&` and `|` (not 'and'/'or') are the standard way to filter rows.
- `isin()`, `between()`, and `query()` make common filtering patterns shorter and more readable.

✗ COMMON MISTAKES

- Using Python's 'and'/'or' instead of `&`/`|` inside a boolean filter.
- Forgetting parentheses around each condition when combining multiple filters.
- Confusing `df.loc[0:2]` (returns 3 rows, inclusive) with `df.iloc[0:2]` (returns 2 rows, exclusive).

★ TRAINER TIP

- Live-demo the exact error from dropping parentheses or using 'and' — students will hit this within their first week, so seeing the traceback now saves debugging time later.

? STUDENT Q&A

Q: When should I use loc vs iloc?

A: Use loc when you know column/row names; use iloc when you only know positions, e.g. 'the first 5 rows'.

Q: Can query() reference a variable from outside the DataFrame?

A: Yes — prefix it with @, e.g. df.query('Age > @min_age').

Q: Is chained indexing like df[df['Age']>22]['City'] safe?

A: It works for reading but is unsafe and slow for writing — use loc for any assignment, e.g. df.loc[df['Age']>22, 'City'].

⚙ INTERVIEW INSIGHT

'What is the difference between loc and iloc?' is one of the most frequently asked Pandas interview questions — always mention the inclusive/exclusive slicing distinction, not just label vs. position.

6. Data Cleaning

This is where `students_raw.csv` earns its name. We'll clean it step by step and, by the end of the section, save the result as `students_clean.csv` — the file used for every section from here on.

6.1 Missing Values

Real datasets are rarely complete. Pandas represents missing data as `NaN` (Not a Number) for numeric columns and `None`/`NaT` for objects and dates.

```
df = pd.read_csv('students_raw.csv')
print(df.isnull().sum())    # count of missing values per column
```

Output:

```
StudentID    0
Name          0
Gender        0
City          0
JoinDate      0
Math          3
Science       2
English       3
dtype: int64
```

`dropna()`

```
print(df.dropna().shape)           # drop rows with ANY missing value
print(df.dropna(subset=['Math']).shape)  # drop only where Math is missing
```

Output:

```
(33, 8)
(38, 8)
```

`fillna()`

```
df['Math'] = df['Math'].fillna(df['Math'].mean())
print(df[['StudentID', 'Math']].head(6))
```

Output:

```
StudentID    Math
0          1  95.000000
1          2  62.000000
2          3  45.000000
3          4  67.315789  <- was NaN, now filled with the column mean (67.32)
4          5  75.000000
5          6  79.000000
```

BEST PRACTICE

Dropping rows loses data permanently — prefer `fillna()` with a sensible value (mean/median for numeric, mode or 'Unknown' for categorical) unless the missing rate is very small.

6.2 `replace()`

```
df['Gender'] = df['Gender'].replace({'M': 'Male', 'F': 'Female'})
print(df['Gender'].head(4))
```

Output:

```
0      Male
1    Female
2    Female
3      Male
Name: Gender, dtype: object
```

6.3 rename()

```
df = df.rename(columns={'StudentID': 'Student_ID'})
print(df.columns)
```

Output:

```
Index(['Student_ID', 'Name', 'Gender', 'City', 'JoinDate', 'Math', 'Science',
      'English'],
      dtype='object')
```

6.4 Duplicate Rows

```
print(df.duplicated().sum())      # 1 duplicate row was deliberately included
df = df.drop_duplicates()
print(df.shape)
```

Output:

```
1
(40, 8)
```

6.5 astype() — Fixing Data Types

```
df['JoinDate'] = df['JoinDate'].astype('datetime64[ns]')
print(df['JoinDate'].dtype)
```

Output:

```
datetime64[ns]
```

6.6 String Cleaning

Recall from Section 4 that City had 7 unique values instead of 5, purely due to inconsistent casing ('MUMBAI', 'delhi') and Name had leading whitespace (' Arjun'). Let's fix both:

```
df['Name'] = df['Name'].str.strip().str.title()      # remove whitespace, normalize case
df['City'] = df['City'].str.strip().str.title()
print(df['City'].unique())
```

Output:

```
['Delhi' 'Chennai' 'Mumbai' 'Kolkata' 'Pune']      <- back down to the correct 5 cities
```

Saving the Cleaned File

```
df.to_csv('students_clean.csv', index=False)
print('students_clean.csv saved:', df.shape)
```

Output:

```
students_clean.csv saved: (40, 8)
```

FROM HERE ON

Sections 7, 8, and 10 all load `students_clean.csv` directly, so you're always working with the tidy 40-row version from this point forward.

✓ KEY TAKEAWAYS

- `isnull()/notnull()` detect missing data; `dropna()/fillna()` decide how to handle it.
- `drop_duplicates()` and `astype()` are essential before any aggregation to avoid inflated counts or type errors.
- The `.str` accessor unlocks vectorized string cleaning — `strip`, `lower`, `replace`, `contains`, and more.

✗ COMMON MISTAKES

- Calling `dropna()` blindly, silently losing large portions of the dataset.
- Filling missing numeric values with 0 when a mean/median would be more representative.
- Forgetting the `.str` accessor and trying to call `.strip()` directly on a Series (raises `AttributeError`).

★ TRAINER TIP

- Show `df.isnull().sum()` as a mandatory checkpoint immediately after every `read_csv()` — make it muscle memory before moving to any analysis.

? STUDENT Q&A

Q: What's the difference between `None` and `NaN` in Pandas?

A: `None` is Python's generic null; Pandas converts it to `NaN` (float) in numeric columns and `NaT` in datetime columns for consistency.

Q: Does `dropna()` modify the original `DataFrame`?

A: No, by default it returns a new `DataFrame` — pass `inplace=True` to modify in place, or reassign `df = df.dropna()`.

Q: How do I remove duplicates based on only some columns?

A: Pass `subset=['col1','col2']` to `drop_duplicates()`.

🔍 INTERVIEW INSIGHT

A frequent scenario question: 'You have a column that's 30% missing — what do you do?' Strong answers discuss the missingness pattern (random vs. systematic) before jumping to `fillna` or `dropna`.

7. Data Manipulation

From here on we work with `students_clean.csv` — the tidy 40-row output of Section 6 (missing marks filled, duplicate removed, Name/City cleaned).

```
df = pd.read_csv('students_clean.csv')
```

7.1 Sorting

```
print(df.sort_values('Math', ascending=False)[['StudentID', 'Name', 'Math']].head(5))
```

Output:

	StudentID	Name	Math
0	1	Aditi	95.0
14	15	Neha	92.0
9	10	Rahul	88.0
12	13	Ishita	88.0
20	21	Ritu	88.0

```
print(df.sort_values(['City', 'Math'], ascending=[True, False])[['Name', 'City', 'Math']].head(5))
```

Output:

	Name	City	Math
9	Rahul	Chennai	88.0
8	Anjali	Chennai	84.0
18	Tanvi	Chennai	75.0
34	Isha	Chennai	67.0
1	Rohan	Chennai	62.0

7.2 Adding, Updating and Deleting Columns

```
df['Pass'] = df['Math'] >= 40 # add a new derived column
print(df[['StudentID', 'Math', 'Pass']].head(5))
```

Output:

	StudentID	Math	Pass
0	1	95.0	True
1	2	62.0	True
2	3	45.0	True
3	4	66.6	True
4	5	75.0	True

```
df['Total'] = df['Math'] + df['Science'] + df['English'] # add a computed column
df = df.drop(columns=['Pass']) # delete a column
print(df[['StudentID', 'Math', 'Science', 'English', 'Total']].head(5))
```

Output:

	StudentID	Math	Science	English	Total
0	1	95.0	63.0	98.0	256.0
1	2	62.0	77.0	77.0	216.0
2	3	45.0	76.0	61.0	182.0
3	4	66.6	65.0	61.0	192.6

4	5	75.0	89.0	61.0	225.0
---	---	------	------	------	-------

7.3 apply(), map() and applymap()

Function	Works On	Use Case
Series.map(func)	Single Series	Element-wise transform / lookup replace
Series/DataFrame.apply(func)	Series or rows/columns of a DataFrame	Custom row-wise or column-wise logic
DataFrame.applymap(func) *	Every cell of a DataFrame	Element-wise transform across all cells

```
df['Grade'] = df['Math'].map(lambda m: 'A'
if m >= 80 else ('B' if m >= 60 else 'C'))
print(df[['StudentID', 'Math',
'Grade']].head(6))
```

Output:

	StudentID	Math	Grade
0	1	95.0	A
1	2	62.0	B
2	3	45.0	C
3	4	66.6	B
4	5	75.0	B
5	6	79.0	B

```
df['RowTotal'] = df[['Math', 'Science',
'English']].apply(sum, axis=1) # row-
wise
print(df[['StudentID',
'RowTotal']].head(4))
```

Output:

	StudentID	RowTotal
0	1	256.0
1	2	216.0
2	3	182.0
3	4	192.6

```
boosted = df[['Math',
'Science']].apply(lambda col: col + 1) #
element-wise, whole columns
print(boosted.head(3))
```

Output:

Function	Works On	Use Case												
<table border="1"> <thead> <tr> <th></th><th>Math</th><th>Science</th></tr> </thead> <tbody> <tr> <td>0</td><td>96.0</td><td>64.0</td></tr> <tr> <td>1</td><td>63.0</td><td>78.0</td></tr> <tr> <td>2</td><td>46.0</td><td>77.0</td></tr> </tbody> </table>		Math	Science	0	96.0	64.0	1	63.0	78.0	2	46.0	77.0		
	Math	Science												
0	96.0	64.0												
1	63.0	78.0												
2	46.0	77.0												

NOTE

`applymap()` is deprecated as of Pandas 2.1 in favor of `DataFrame.map()` — mention both, but teach `df.map()` as the forward-looking syntax.

7.4 Lambda Functions in Pandas

```
df['Performance'] = df['Math'].apply(lambda x: 'Strong' if x >= 75 else 'Needs Support')
print(df[['StudentID', 'Math', 'Performance']].head(4))
```

Output:

	StudentID	Math	Performance
0	1	95.0	Strong
1	2	62.0	Needs Support
2	3	45.0	Needs Support
3	4	66.6	Needs Support

7.5 Vectorized Operations

A vectorized operation applies to an entire column at once (in fast, compiled C code) instead of looping row-by-row in Python.

```
# Slow: Python loop
totals = []
for m, s, e in zip(df['Math'], df['Science'], df['English']):
    totals.append(m + s + e)

# Fast: vectorized (same result, no explicit loop)
df['Total'] = df['Math'] + df['Science'] + df['English']
print(df['Total'].head(5))
```

Output:

```
0    256.0
1    216.0
2    182.0
3    192.6
4    225.0
Name: Total, dtype: float64
```

PERFORMANCE

The vectorized version above is typically 20-100x faster than the Python loop for large datasets because it avoids the per-row overhead of the Python interpreter.

✓ KEY TAKEAWAYS

- `sort_values()` and `sort_index()` cover virtually all sorting needs, including multi-column sorts.
- `map()` is for simple element-wise lookups; `apply()` handles more complex row/column-wise logic.
- Vectorized operations (`df['a'] + df['b']`) should always be preferred over explicit Python loops.

✗ COMMON MISTAKES

- Using `.apply(lambda row: row['a']+row['b'], axis=1)` when a direct vectorized `df['a']+df['b']` would be simpler and far faster.
- Forgetting `axis=1` when applying a function row-wise (default `axis=0` applies column-wise).
- Chaining too many lambda functions instead of a named function, hurting readability.

★ TRAINER TIP

- Time both the loop and vectorized version live with `%timeit` in a notebook — seeing '2.3s vs 4ms' on screen makes the vectorization lesson unforgettable.

? STUDENT Q&A

Q: When should I actually use `apply()` instead of vectorization?

A: When the logic genuinely can't be expressed as simple arithmetic/boolean operations — e.g., calling an external function or complex conditional branching.

Q: Is `apply()` itself vectorized?

A: No — under the hood it still loops row by row in Python; it's more convenient than a manual loop, but not faster.

Q: What's the fastest way to create a conditional column?

A: `np.where()` or `np.select()` for simple/multiple conditions — both are faster than `apply()`.

⚙️ INTERVIEW INSIGHT

'Why is vectorization faster than a for loop in Pandas?' tests real understanding — a good answer mentions avoiding per-element Python interpreter overhead by operating on contiguous NumPy memory in C.

8. GroupBy and Aggregation

`groupby()` is the single most powerful tool in Pandas for business reporting — it implements the classic 'split → apply → combine' pattern. We'll use `students_clean.csv` again, plus `sales.csv` (180 rows of store sales) for a second, business-flavored example.

SPLIT	APPLY	COMBINE
group by 'City'	compute <code>mean(Math)</code>	merge results back
Delhi rows	----> avg = 73.6	---->
Pune rows	----> avg = 65.0	----> City Math
Mumbai rows	----> avg = 68.9	----> Delhi 73.6
		Pune 65.0
		Mumbai 68.9

8.1 Basic `groupby()`

```
df = pd.read_csv('students_clean.csv')
print(df.groupby('City')['Math'].mean().round(2))
```

Output:

```
City
Chennai    60.64
Delhi       73.58
Kolkata     68.00
Mumbai      68.94
Pune        64.96
Name: Math, dtype: float64
```

```
print(df.groupby('City')['Math'].agg(['count', 'min', 'max', 'median', 'std']).round(2))
```

Output:

```

      count  min  max  median  std
City
Chennai    11  42.0  88.0    62.0  16.46
Delhi       8  53.0  95.0    70.8  13.95
Kolkata     5  35.0  88.0    79.0  22.70
Mumbai      7  37.0  88.0    77.0  21.12
Pune        9  41.0  92.0    66.6  16.79
```

8.2 Multiple Aggregations with `agg()`

```
# Named aggregations (clean, self-documenting column names)
report = df.groupby('City').agg(
    total_math=('Math', 'sum'),
    avg_math=('Math', 'mean'),
    students=('Math', 'count')
).round(2)
print(report)
```

Output:

```

      total_math  avg_math  students
City
Chennai      667.0    60.64         11
Delhi        588.4    73.58          8
Kolkata      592.0    68.00          5
Mumbai      582.5    68.94          7
Pune         604.7    64.96          9
```

Chennai	667.0	60.64	11
Delhi	588.6	73.58	8
Kolkata	340.0	68.00	5
Mumbai	482.6	68.94	7
Pune	584.6	64.96	9

8.3 Grouping by Multiple Columns

```
print(df.groupby(['City', 'Gender'])['Math'].mean().round(2))
```

Output:

City	Gender	
Chennai	F	58.83
	M	62.80
Delhi	F	67.00
	M	80.15
Kolkata	F	64.00
	M	84.00
Mumbai	F	66.00
	M	76.30
Pune	F	71.50
	M	59.72

Name: Math, dtype: float64

Business-Oriented Example (sales.csv)

```
sales = pd.read_csv('sales.csv')

# Total revenue per store, sorted highest to lowest
report = (sales.groupby('Store')['Revenue']
          .sum()
          .reset_index()
          .sort_values('Revenue', ascending=False))
print(report.head())
```

Output:

Store	Revenue
Bengaluru	232850
Pune	206450
Mumbai	183800
Kolkata	171600
Chandigarh	171000

✓ KEY TAKEAWAYS

- `groupby()` follows split-apply-combine: split into groups, apply a function, combine results.
- `agg()` supports multiple functions at once, and named aggregations produce clean column names.
- Grouping by multiple columns creates a hierarchical (MultiIndex) result unless `reset_index()` is used.

✗ COMMON MISTAKES

- Forgetting `reset_index()` after a `groupby`, leaving a MultiIndex that's awkward to merge or export.
- Using `groupby()['col'].sum()` when multiple columns need different aggregations — `agg()` is cleaner.
- Applying `groupby` on the entire DataFrame accidentally, aggregating every column, including irrelevant ones.

★ TRAINER TIP

- Draw the split-apply-combine diagram on the whiteboard before showing any code — students who visualize the mechanism debug groupby errors far more effectively later.

? STUDENT Q&A

Q: Why did my groupby() result come back as a Series, not a DataFrame?

A: Selecting a single column, e.g. `groupby('City')['Sales'].sum()`, returns a Series; select a list of columns to keep a DataFrame.

Q: How do I filter groups, not just rows?

A: Use `groupby().filter(lambda g: condition)` to keep or drop entire groups based on an aggregate condition.

Q: What is a MultiIndex?

A: A hierarchical row index with more than one level, commonly produced when grouping by multiple columns without `reset_index()`.

⚙️ INTERVIEW INSIGHT

A classic scenario question: 'Find the top-selling product per region.' The efficient answer uses `groupby('region').apply()` combined with `sort_values()` and `head(1)`, or `idxmax()` — interviewers look for awareness of both approaches.

9. Merge, Join and Concatenate

Real analytics work rarely uses a single table — customer data, order data, and product data usually live separately and must be combined, just like SQL joins. We'll use two small real files: `customers.csv` (6 customers) and `orders.csv` (7 orders, two of which reference customers who don't exist in `customers.csv` — 107 and 108 — exactly the kind of mismatch you'll hit in practice).

```
customers = pd.read_csv('customers.csv')
orders = pd.read_csv('orders.csv')
print(customers)
print(orders)
```

Output:

	CustID	Name	City
0	101	Ananya Rao	Delhi
1	102	Bilal Khan	Mumbai
2	103	Chitra Iyer	Pune
3	104	Dev Malhotra	Chennai
4	105	Esha Kapoor	Kolkata
5	106	Faisal Ahmed	Delhi

	OrderID	CustID	Product	Amount	OrderDate
0	1001	101	Laptop	55000	2026-01-05
1	1002	102	Headphones	2500	2026-01-08
2	1003	102	Monitor	9800	2026-01-15
3	1004	104	Keyboard	1200	2026-02-02
4	1005	107	Webcam	3200	2026-02-10
5	1006	101	Mouse	700	2026-02-14
6	1007	108	Tablet	21000	2026-03-01

9.1 concat() — Stacking DataFrames

```
jan = orders[orders['OrderDate'] < '2026-02-01']
feb = orders[orders['OrderDate'] >= '2026-02-01']
all_orders = pd.concat([jan, feb], axis=0, ignore_index=True) # stack rows back together
print(all_orders.shape) # (7, 5) - back to the original row count
```

Output:

(7, 5)

9.2 merge() — SQL-Style Joins

```
print(pd.merge(orders, customers, on='CustID', how='inner'))
```

Output:

	OrderID	CustID	Product	Amount	OrderDate	Name	City
0	1001	101	Laptop	55000	2026-01-05	Ananya Rao	Delhi
1	1002	102	Headphones	2500	2026-01-08	Bilal Khan	Mumbai
2	1003	102	Monitor	9800	2026-01-15	Bilal Khan	Mumbai
3	1004	104	Keyboard	1200	2026-02-02	Dev Malhotra	Chennai
4	1006	101	Mouse	700	2026-02-14	Ananya Rao	Delhi

orders 1005 (CustID 107) and 1007 (CustID 108) are dropped - no matching customer

```
print(pd.merge(orders, customers, on='CustID', how='left'))
```

Output:

	OrderID	CustID	Product	Amount	OrderDate	Name	City
0	1001	101	Laptop	55000	2026-01-05	Ananya Rao	Delhi
1	1002	102	Headphones	2500	2026-01-08	Bilal Khan	Mumbai
2	1003	102	Monitor	9800	2026-01-15	Bilal Khan	Mumbai
3	1004	104	Keyboard	1200	2026-02-02	Dev Malhotra	Chennai
4	1005	107	Webcam	3200	2026-02-10	NaN	NaN
5	1006	101	Mouse	700	2026-02-14	Ananya Rao	Delhi
6	1007	108	Tablet	21000	2026-03-01	NaN	NaN

```
print(pd.merge(orders, customers, on='CustID', how='right'))
```

Output:

	OrderID	CustID	Product	Amount	OrderDate	Name	City
0	1001.0	101	Laptop	55000.0	2026-01-05	Ananya Rao	Delhi
1	1006.0	101	Mouse	700.0	2026-02-14	Ananya Rao	Delhi
2	1002.0	102	Headphones	2500.0	2026-01-08	Bilal Khan	Mumbai
3	1003.0	102	Monitor	9800.0	2026-01-15	Bilal Khan	Mumbai
4	NaN	103	NaN	NaN	NaN	Chitra Iyer	Pune
5	1004.0	104	Keyboard	1200.0	2026-02-02	Dev Malhotra	Chennai
6	NaN	105	NaN	NaN	NaN	Esha Kapoor	Kolkata
7	NaN	106	NaN	NaN	NaN	Faisal Ahmed	Delhi

```
print(pd.merge(orders, customers, on='CustID', how='outer'))
```

Output:

	OrderID	CustID	Product	Amount	OrderDate	Name	City
0	1001.0	101	Laptop	55000.0	2026-01-05	Ananya Rao	Delhi
1	1006.0	101	Mouse	700.0	2026-02-14	Ananya Rao	Delhi
2	1002.0	102	Headphones	2500.0	2026-01-08	Bilal Khan	Mumbai
3	1003.0	102	Monitor	9800.0	2026-01-15	Bilal Khan	Mumbai
4	NaN	103	NaN	NaN	NaN	Chitra Iyer	Pune
5	1004.0	104	Keyboard	1200.0	2026-02-02	Dev Malhotra	Chennai
6	NaN	105	NaN	NaN	NaN	Esha Kapoor	Kolkata
7	NaN	106	NaN	NaN	NaN	Faisal Ahmed	Delhi
8	1005.0	107	Webcam	3200.0	2026-02-10	NaN	NaN
9	1007.0	108	Tablet	21000.0	2026-03-01	NaN	NaN

9.3 join() — Merge on the Index

```
print(orders.set_index('CustID').join(customers.set_index('CustID'), how='left').head())
```

Output:

	OrderID	Product	Amount	OrderDate	Name	City
CustID						
101	1001	Laptop	55000	2026-01-05	Ananya Rao	Delhi
102	1002	Headphones	2500	2026-01-08	Bilal Khan	Mumbai
102	1003	Monitor	9800	2026-01-15	Bilal Khan	Mumbai
104	1004	Keyboard	1200	2026-02-02	Dev Malhotra	Chennai
107	1005	Webcam	3200	2026-02-10	NaN	NaN

9.4 Join Types Explained

Join Type	Keeps	SQL Equivalent
inner	Only matching rows in both tables	INNER JOIN
left	All rows from the left table + matches	LEFT JOIN
right	All rows from the right table + matches	RIGHT JOIN
outer	All rows from both, matched where possible	FULL OUTER JOIN

INNER JOIN	LEFT JOIN
RIGHT JOIN	OUTER JOIN
+---+---+	+---+---+
+---+---+	+---+---+
A B	A B?
A? B	A? B?
+---+---+	+---+---+
+---+---+	+---+---+
(only overlap)	(all of A)
(all of B)	(all of A & B)

concat() vs merge() vs join()

Function	Purpose	Aligns By
concat()	Stack DataFrames vertically or side-by-side	Index or column position
merge()	SQL-style join on one or more key columns	Column values (keys)
join()	Merge convenience method	Index (by default)

Common Interview Questions

- Q: What happens to unmatched rows in an inner join? A: They are dropped entirely from the result.
- Q: What's the difference between merge() and join()? A: join() merges on the index by default; merge() merges on column keys by default (though it can also use indexes).
- Q: How do you merge on columns with different names in each table? A: Use left_on and right_on instead of on.

✓ KEY TAKEAWAYS

- concat() stacks DataFrames; merge() and join() combine them based on shared keys, SQL-style.
- how='inner'/'left'/'right'/'outer' controls which rows survive when keys don't perfectly match.
- join() is a convenience wrapper around merge() that defaults to joining on the index.

✗ COMMON MISTAKES

- Using concat() when a key-based merge() was actually needed, silently misaligning rows.
- Not checking for duplicate keys before merging, which can unexpectedly multiply row counts (a 'fan-out').
- Forgetting how='left' when unmatched rows from the primary table must be preserved.

★ TRAINER TIP

- Use a tiny 3-row orders table and 3-row customers table with one deliberate mismatch, and run all four join types live side-by-side so students see the row-count differences directly.

? STUDENT Q&A

Q: How do I know if a merge accidentally duplicated rows?

A: Compare `len(result)` to `len(left)` before and after; a jump usually means duplicate keys on one side.

Q: Can I merge on multiple columns at once?

A: Yes — pass a list to 'on', e.g. `on=['CustID','Region']`.

Q: What's the indicator parameter for?

A: `merge(..., indicator=True)` adds a `'_merge'` column showing whether each row came from `'left_only'`, `'right_only'`, or `'both'` — great for auditing joins.

🔗 INTERVIEW INSIGHT

A go-to interview scenario: 'You merged two tables and got more rows than expected — why?' The expected answer: duplicate keys on one or both sides cause a Cartesian-style fan-out for each match.

10. Working with Date and Time

This section uses `sales.csv`, which has an `OrderDate` column loaded as plain text (like every CSV date column, before you fix it).

10.1 `to_datetime()`

```
sales = pd.read_csv('sales.csv')
print(sales['OrderDate'].dtype)           # object - still just text
sales['OrderDate'] = pd.to_datetime(sales['OrderDate'])
print(sales['OrderDate'].dtype)           # datetime64[ns] - now a real date
```

Output:

```
object
datetime64[ns]
```

10.2 Current Date and Time

```
today = pd.Timestamp.today()
now = pd.Timestamp.now()
print(today)
print(now)
```

Output:

```
2026-07-21 00:00:00
2026-07-21 14:32:07.881452
```

10.3 Extracting Date Parts

```
sales['Year'] = sales['OrderDate'].dt.year
sales['Month'] = sales['OrderDate'].dt.month
sales['Day'] = sales['OrderDate'].dt.day
sales['Weekday'] = sales['OrderDate'].dt.day_name()
print(sales[['OrderDate', 'Year', 'Month', 'Day', 'Weekday']].head(3))
```

Output:

	OrderDate	Year	Month	Day	Weekday
0	2026-01-07	2026	1	7	Wednesday
1	2026-01-07	2026	1	7	Wednesday
2	2026-01-09	2026	1	9	Friday

10.4 Date Filtering

```
print(sales[sales['OrderDate'] >= '2026-03-01'][['OrderDate', 'Store', 'Revenue']].head(4))
```

Output:

	OrderDate	Store	Revenue
58	2026-03-02	Bengaluru	15600
59	2026-03-02	Pune	3600
60	2026-03-03	Bengaluru	9600

```
61 2026-03-04    Mumbai    2400
```

```
print(sales[sales['OrderDate'].dt.month == 1][['OrderDate', 'Store', 'Revenue']].head(4))
```

Output:

	OrderDate	Store	Revenue
0	2026-01-07	Patna	900
1	2026-01-07	Chandigarh	600
2	2026-01-09	Chandigarh	18000
3	2026-01-11	Patna	2850

10.5 Date Arithmetic

```
reference_date = pd.Timestamp('2026-07-01')
sales['DaysSinceOrder'] = (reference_date - sales['OrderDate']).dt.days
sales['DueDate'] = sales['OrderDate'] + pd.Timedelta(days=30)
print(sales[['OrderDate', 'DaysSinceOrder', 'DueDate']].head(3))
```

Output:

	OrderDate	DaysSinceOrder	DueDate
0	2026-01-07	175	2026-02-06
1	2026-01-07	175	2026-02-06
2	2026-01-09	173	2026-02-08

```
print(sales['OrderDate'].min(), '->', sales['OrderDate'].max())
```

Output:

```
2026-01-07 00:00:00 -> 2026-06-29 00:00:00
```

✓ KEY TAKEAWAYS

- `to_datetime()` converts text into a proper `datetime64` dtype, unlocking the entire `.dt` accessor.
- The `.dt` accessor extracts year, month, day, weekday, and more from a datetime column.
- `Timedelta` enables clean date arithmetic — adding/subtracting days, computing durations.

✗ COMMON MISTAKES

- Comparing date strings without converting to datetime first, producing incorrect string-based sorting.
- Forgetting the `.dt` accessor and trying to call `.year` directly on a Series.
- Not specifying `format=` for ambiguous date strings (e.g., 01-02-2026), risking day/month mix-ups.

★ TRAINER TIP

- Show a date column read from CSV as plain text (dtype object) first — students need to see the 'before' state to appreciate what `to_datetime()` actually fixes.

? STUDENT Q&A

Q: Why do I need `dt.year` instead of just `.year`?

A: The `.dt` accessor is required to reach datetime-specific properties on a Series, similar to `.str` for text.

Q: How do I handle multiple date formats in the same column?

A: Use `pd.to_datetime(col, errors='coerce')` to convert what it can and set unparseable values to `NaT` for review.

Q: What's the difference between `Timestamp` and `Timedelta`?

A: Timestamp represents a specific point in time; Timedelta represents a duration or difference between two Timestamps.

⚙️ INTERVIEW INSIGHT

Interviewers often ask how to calculate 'days since last purchase' per customer — a strong answer combines `groupby('CustomerID')['OrderDate'].max()` with today's date and `.dt.days`.

© Shalinee Priya

11. Performance and Optimization

11.1 Python Lists vs. NumPy vs. Pandas

	Python List	NumPy Array	Pandas Series/DataFrame
Type	Heterogeneous	Homogeneous	Column-wise homogeneous
Speed (bulk math)	Slow (loop)	Fast (vectorized C)	Fast (built on NumPy)
Labels	No	No	Yes (Index/Columns)
Missing value handling	Manual	Limited	Built-in (NaN, isna)
Best for	General-purpose Python	Numeric arrays	Labeled tabular data

11.2 Vectorization

As covered in Section 7, vectorized operations delegate looping to optimized, compiled C code inside NumPy/Pandas, avoiding the overhead of Python's interpreter for each element.

11.3 Memory Usage

```
df.info(memory_usage='deep')
df['City'] = df['City'].astype('category') # big memory saving for repeated strings
```

TIP

Converting low-cardinality text columns (like City or Gender) to the 'category' dtype can cut memory usage by 70-90% on large datasets.

11.4 Execution Speed: A Benchmark

```
import time
n = 1_000_000
s = pd.Series(range(n))

start = time.time()
result_loop = [x * 2 for x in s] # Python-level loop
print('Loop:', time.time() - start)

start = time.time()
result_vec = s * 2 # vectorized
print('Vectorized:', time.time() - start)
```

Output:

```
Loop:      0.185 seconds
Vectorized: 0.004 seconds
# Vectorized version is roughly 45x faster
```

11.5 Efficient Filtering & Avoiding Loops

- Prefer boolean masking (`df[df['x']>5]`) over `iterrows()` — `iterrows()` is one of the slowest ways to touch a DataFrame
- Use vectorized string methods (`.str.contains`) instead of looping with Python's 'in' operator
- Use `np.where()` / `np.select()` for conditional column creation instead of `apply()` with a lambda
- Pre-filter columns with `usecols` in `read_csv()` to avoid loading unnecessary data into memory

✓ KEY TAKEAWAYS

- NumPy and Pandas are fast because they vectorize operations in compiled C, not because 'Pandas is inherently faster than Python.'
- Converting repetitive text columns to 'category' dtype is a simple, high-impact memory optimization.
- `iterrows()` and `apply()` with row-wise lambdas are almost always the slowest option — avoid them when a vectorized alternative exists.

✗ COMMON MISTAKES

- Reaching for `iterrows()` out of habit from list-based thinking, instead of a vectorized operation.
- Loading an entire wide CSV when only a handful of columns are actually needed.
- Ignoring dtype optimization on very large datasets, leading to unnecessary memory pressure.

★ TRAINER TIP

- Run the benchmark code live and let students call out guesses for the speed difference before revealing the actual numbers — it turns a dry topic into an engaging moment.

? STUDENT Q&A

Q: Is Pandas always faster than a Python loop?

A: Only for vectorizable operations. For genuinely row-dependent custom logic, a well-written loop can occasionally compete — but vectorization wins the vast majority of the time.

Q: When should I use NumPy directly instead of Pandas?

A: For pure numeric array math without the need for labels, NumPy has slightly less overhead — but the difference rarely matters outside very tight loops.

Q: What's the single best performance habit to build?

A: Never write a for loop over DataFrame rows before checking whether a vectorized or groupby-based alternative exists.

⚙️ INTERVIEW INSIGHT

'How would you speed up a slow Pandas pipeline?' is a common senior-level question. Strong answers mention vectorization, category dtypes, reading only needed columns, and avoiding `iterrows()`/`apply()` where possible.

12. Practical Hands-on Demo (40 Minutes)

Project: Student Performance Analytics System

This is the anchor exercise of the workshop. It walks through a complete, realistic analytics pipeline from raw messy data to a polished, exported report — exactly the workflow used in real analyst jobs. This project uses its own dedicated file, `student_performance_project.csv` (61 rows, deliberately messy: missing marks, one duplicate row), so it feels like a fresh capstone rather than a repeat of earlier sections.

TRAINER TIP

Type this live, step by step, rather than pasting the finished script. Pause after each step and ask 'what do you expect the output to look like?' before running it.

Step 1 — Load the Dataset

```
import pandas as pd
import numpy as np

df = pd.read_csv('student_performance_project.csv')
print(df.shape)
print(df.head())
```

Output:

(61, 7)

	StudentID	Name	Gender	City	Math	Science	English
0	1	Aditi Sharma	F	Bengaluru	33.0	33.0	74.0
1	2	Rohan Verma	F	Mumbai	78.0	97.0	60.0
2	3	Meera Nair	F	Delhi	90.0	81.0	70.0
3	4	Karan Mehta	F	Delhi	45.0	41.0	70.0
4	5	Sneha Rao	M	Delhi	91.0	80.0	51.0

Step 2 — First Look at Data Quality

```
print(df.isnull().sum())
print('Duplicate rows:', df.duplicated().sum())
```

Output:

```
StudentID    0
Name         0
Gender       0
City         0
Math         4
Science      3
English      3
dtype: int64
Duplicate rows: 1
```

Step 3 — Cleaning Missing Values

```
for subject in ['Math', 'Science', 'English']:
```

```
df[subject] = df[subject].fillna(df[subject].mean()).round(1)

print(df.isnull().sum())    # confirm zero missing values remain
```

Output:

```
StudentID    0
Name         0
Gender       0
City         0
Math         0
Science      0
English      0
dtype: int64
```

Step 4 — Removing Duplicates

```
df = df.drop_duplicates(subset=['Name', 'City'], keep='first')
print(df.shape)
```

Output:

```
(60, 7)
```

Step 5 — Filtering Students

```
# Students scoring above 80 in Math
top_math = df[df['Math'] > 80]
print('Students above 80 in Math:', len(top_math))

# Students from Delhi with Science >= 70
delhi_science = df[(df['City'] == 'Delhi') & (df['Science'] >= 70)]
print(delhi_science[['StudentID', 'Name', 'City', 'Science']])
```

Output:

```
Students above 80 in Math: 13
   StudentID  Name  City  Science
2          3  Meera Nair  Delhi    81.0
4          5  Sneha Rao  Delhi    80.0
```

Step 6 — Subject-wise Analysis

```
subject_avg = df[['Math', 'Science', 'English']].mean().round(2)
print(subject_avg)
```

Output:

```
Math        62.47
Science     63.11
English     63.29
dtype: float64
```

Step 7 — Total, Average and Ranking


```
df['Total'] = df[['Math', 'Science', 'English']].sum(axis=1).round(1)
df['Average'] = df[['Math', 'Science', 'English']].mean(axis=1).round(2)
df['Rank'] = df['Total'].rank(ascending=False, method='min').astype(int)
df = df.sort_values('Rank')
print(df[['Rank', 'StudentID', 'Name', 'Total', 'Average']].head(6))
```

Output:

	Rank	StudentID	Name	Total	Average
51	1	52	Lavanya Menon	287.0	95.67
47	2	48	Hitesh Goyal	279.0	93.00
37	3	38	Sameer Ansari	260.0	86.67
2	4	3	Meera Nair	241.0	80.33
52	5	53	Mohit Ahluwalia	238.5	79.50

Step 8 — Pass / Fail Analysis

```
PASS_MARK = 40
df['Result'] = np.where(
    (df['Math'] >= PASS_MARK) & (df['Science'] >= PASS_MARK) & (df['English'] >= PASS_MARK),
    'Pass', 'Fail'
)
print(df['Result'].value_counts())
```

Output:

```
Result
Pass    40
Fail    20
Name: count, dtype: int64
```

Step 9 — Gender-wise and City-wise Analysis

```
gender_report = df.groupby('Gender')['Average'].mean().round(2)
city_report = df.groupby('City')['Average'].mean().round(2)
print(gender_report)
print(city_report)
```

Output:

```
Gender
F    63.89
M    61.74
Name: Average, dtype: float64
```

```
City
Bengaluru    54.37
Chennai      67.59
Delhi        66.35
Kolkata      58.30
Mumbai       63.84
Pune         66.32
Name: Average, dtype: float64
```

Step 10 — Combined GroupBy Report

```
summary = df.groupby(['City', 'Gender']).agg(
    students=('StudentID', 'count'),
    avg_total=('Total', 'mean'),
    pass_rate=('Result', lambda x: (x == 'Pass').mean() * 100)
).round(1).reset_index()
print(summary)
```

Output:

	City	Gender	students	avg_total	pass_rate
0	Bengaluru	F	5	168.2	60.0
1	Bengaluru	M	3	154.7	0.0
2	Chennai	F	6	205.2	66.7
3	Chennai	M	5	199.9	80.0
4	Delhi	F	5	197.5	100.0
5	Delhi	M	3	201.7	100.0
6	Kolkata	F	3	159.3	0.0
7	Kolkata	M	8	180.8	50.0
8	Mumbai	F	10	189.6	80.0
9	Mumbai	M	3	198.0	100.0
10	Pune	F	5	216.7	80.0
11	Pune	M	4	176.8	50.0

READ THE REPORT

Bengaluru males show a 0% pass rate from just 3 students — a small-sample flag worth calling out live: is this a real trend or too few students to conclude anything? Great moment to discuss statistical significance with the class.

Step 11 — Topper List

```
topper_list = df.sort_values('Total', ascending=False)[['Rank', 'Name', 'City', 'Total',
'Average', 'Result']]
print(topper_list.head(5))
```

Output:

	Rank	Name	City	Total	Average	Result
51	1	Lavanya Menon	Pune	287.0	95.67	Pass
47	2	Hitesh Goyal	Pune	279.0	93.00	Pass
37	3	Sameer Ansari	Chennai	260.0	86.67	Pass
2	4	Meera Nair	Delhi	241.0	80.33	Pass
52	5	Mohit Ahluwalia	Chennai	238.5	79.50	Pass

Step 12 — Exporting the Final Report

```
df.to_csv('student_performance_report.csv', index=False)
df.to_excel('student_performance_report.xlsx', sheet_name='Report', index=False)
print('Report exported successfully:', df.shape)
```

Output:

Report exported successfully: (60, 11)

EXTENSION IDEA

Challenge fast finishers to add a bar chart of average marks per city using `df.groupby('City')['Average'].mean().plot(kind='bar')` with Matplotlib — a natural bridge into the visualization ecosystem covered in the Learning Roadmap.

✓ KEY TAKEAWAYS

- This single project touches nearly every skill from the workshop: I/O, cleaning, filtering, aggregation, ranking, and export.
- `np.where()` is an efficient way to build a conditional Pass/Fail column without `apply()`.
- `groupby().agg()` with named aggregations produces business-ready summary tables in one step.

✗ COMMON MISTAKES

- Filling missing values with the overall mean before checking whether it should be subject-specific or group-specific (e.g., mean per city).
- Forgetting `axis=1` when summing across Math/Science/English for a per-student Total.
- Ranking with the default method, which can produce non-intuitive tie-handling — 'min' is usually most natural for leaderboards.

★ TRAINER TIP

- Let students discover the duplicate row and missing values themselves via `isnull().sum()` and `duplicated().sum()` in Step 2, rather than pointing them out — discovery reinforces the habit far better than being told.

? STUDENT Q&A

Q: Why fillna with the column mean instead of an overall constant?

A: It preserves the underlying distribution of each subject far more faithfully than an arbitrary constant like zero.

Q: What does `rank(method='min')` actually do with ties?

A: Tied values receive the same (lowest) rank, and the next rank skips ahead accordingly — e.g., two students tied for 1st both get rank 1, and the next student gets rank 3.

Q: How would I extend this to handle 5,000 students instead of 60?

A: No code changes needed — every operation here is fully vectorized and scales automatically.

⚙️ INTERVIEW INSIGHT

This exact style of project — 'build an end-to-end report from a messy dataset' — is a very common take-home assignment in analyst interviews. Being able to narrate this pipeline fluently is a strong signal of real competence.

13. Mini Exercises

13.1 Practice Questions (20)

Quick-fire conceptual questions — ideal for a rapid round-robin review at the end of class.

1. What is the difference between a Series and a DataFrame?
2. How do you check the number of rows and columns in a DataFrame?
3. Which function shows the data type of every column at once?
4. How do you select only the 'Name' and 'Age' columns from a DataFrame?
5. What is the difference between `df.loc[]` and `df.iloc[]`?
6. How do you filter rows where Age is greater than 25?
7. Why do you need parentheses when combining conditions with `&` or `|`?
8. How do you find how many missing values exist in each column?
9. What is the difference between `dropna()` and `fillna()`?
10. How do you remove duplicate rows based on a single column?
11. Which method converts a column's data type to integer?
12. How do you sort a DataFrame by two columns at once?
13. What is the purpose of the `apply()` function?
14. What does `groupby()` actually do, conceptually?
15. What is the difference between `merge()` and `concat()`?
16. Name the four types of joins available in `merge()`.
17. How do you extract just the year from a datetime column?
18. Why is vectorization faster than a Python for loop?
19. What does the category dtype optimize for?
20. How do you export a DataFrame to an Excel file without the index column?

13.2 Coding Exercises (10) — With Solutions

Exercise 1

Create a Series of 5 city names with a custom index of A, B, C, D, E.

```
# Solution
pd.Series(['Delhi', 'Mumbai', 'Pune', 'Chennai', 'Kolkata'], index=list('ABCDE'))
```

Exercise 2

Create a DataFrame from a dictionary of 3 students with Name and Marks columns.

```
# Solution
pd.DataFrame({'Name': ['A', 'B', 'C'], 'Marks': [80, 90, 70]})
```

Exercise 3

Read a CSV file and print only the first 3 rows.

```
# Solution
pd.read_csv('file.csv').head(3)
```

Exercise 4

Filter a DataFrame for rows where 'Marks' is between 60 and 90.

```
# Solution
df[df['Marks'].between(60, 90)]
```

Exercise 5

Fill missing values in a 'Score' column with its median.

```
# Solution
df['Score'] = df['Score'].fillna(df['Score'].median())
```

Exercise 6

Rename a column 'Stu_Name' to 'Name'.

```
# Solution
df = df.rename(columns={'Stu_Name': 'Name'})
```

Exercise 7

Group a sales DataFrame by 'Region' and compute total revenue.

```
# Solution
df.groupby('Region')['Revenue'].sum()
```

Exercise 8

Merge two DataFrames on a common 'ID' column using a left join.

```
# Solution
pd.merge(df1, df2, on='ID', how='left')
```

Exercise 9

Add a new column 'Year' extracted from a datetime column 'JoinDate'.

```
# Solution
df['Year'] = df['JoinDate'].dt.year
```

Exercise 10

Convert a 'City' column to the category dtype to save memory.

```
# Solution
df['City'] = df['City'].astype('category')
```

13.3 Debugging Exercises (5) — With Solutions

Each snippet below contains a real, common bug. Ask students to spot the issue before revealing the fix.

Debug Challenge 1

```
df[df['Age'] > 20 and df['City'] == 'Pune'] -> raises a ValueError
```

FIX

Use & instead of 'and', and wrap each condition in parentheses: `df[(df['Age'] > 20) & (df['City'] == 'Pune')]`

Debug Challenge 2

```
df['Name'].strip() -> raises AttributeError: 'Series' object has no attribute 'strip'
```

FIX

Use the .str accessor for string methods on a Series: `df['Name'].str.strip()`

Debug Challenge 3

```
df.dropna() -> the DataFrame looks unchanged after this line
```

FIX

`dropna()` returns a new DataFrame by default; reassign it: `df = df.dropna()`, or pass `inplace=True`.

Debug Challenge 4

```
df.groupby('City').sum() -> raises a TypeError on a text column
```

FIX

Select only the numeric column(s) before aggregating: `df.groupby('City')['Sales'].sum()`, or pass `numeric_only=True`.

Debug Challenge 5

```
pd.merge(orders, customers, on='CustID') -> row count is much higher than expected
```

FIX

One of the tables has duplicate `CustID` values, causing a fan-out. Check with `df['CustID'].duplicated().sum()` before merging.

✓ KEY TAKEAWAYS

- The practice questions reinforce vocabulary and conceptual understanding fast.
- Coding exercises build muscle memory for the syntax patterns used daily.
- Debugging exercises train the error-reading skill that separates confident practitioners from copy-paste coders.

✗ COMMON MISTAKES

- Skipping the debugging exercises — reading tracebacks is a skill that needs deliberate practice.
- Rushing through coding exercises without typing them by hand.

★ TRAINER TIP

- Run the debugging exercises as a live group challenge with a 60-second timer per bug — competitive energy keeps the room engaged in the final stretch of a 3-hour session.

? STUDENT Q&A

Q: Can I use these as homework?

A: Yes — they're designed to be self-contained and solvable with only what's covered in this manual.

Q: Where can I get more practice datasets?

A: Kaggle Datasets and the seaborn built-in sample datasets (`sns.load_dataset`) are excellent free sources.

⚙ INTERVIEW INSIGHT

Live-coding rounds in interviews often mirror exactly this format — a short, realistic bug to find and fix under time pressure, rather than an abstract algorithm question.

14. Interview Questions (30) — Beginner to Advanced

These 30 questions mirror what candidates are actually asked in Pandas / data-analyst interviews, organized by difficulty, plus a scenario-based set for senior-style rounds.

14.1 Beginner (1–10)

#	Question	Answer
1	What is Pandas used for?	Reading, cleaning, transforming, and analyzing structured/tabular data in Python.
2	What are the two main data structures in Pandas?	Series (1-D) and DataFrame (2-D).
3	How do you install Pandas?	<code>pip install pandas</code>
4	How do you read a CSV file into a DataFrame?	<code>pd.read_csv('file.csv')</code>
5	What does <code>df.head()</code> do?	Returns the first 5 rows by default (or <code>n</code> if specified).
6	What does <code>df.shape</code> return?	A tuple of (number of rows, number of columns).
7	How do you select a single column?	<code>df['column_name']</code> , which returns a Series.
8	What is NaN in Pandas?	A special float value representing missing/undefined numeric data.
9	How do you check for missing values?	<code>df.isnull()</code> or <code>df.isnull().sum()</code> for per-column counts.
10	What does <code>df.describe()</code> return?	Summary statistics (count, mean, std, min, quartiles, max) for numeric columns.

14.2 Intermediate (11–20)

#	Question	Answer
1	What is the difference between <code>loc[]</code> and <code>iloc[]</code> ?	<code>loc</code> is label-based and inclusive of the end of a slice; <code>iloc</code> is integer-position-based and exclusive of the end.
2	How do you handle missing data in Pandas?	Detect with <code>isnull()</code> ; then either <code>dropna()</code> to remove or <code>fillna()</code> to impute (mean/median/mode/constant/forward-fill).
3	What is the difference between <code>merge()</code> and <code>concat()</code> ?	<code>merge()</code> joins DataFrames on key columns (like SQL joins); <code>concat()</code> stacks DataFrames along an axis without key-based matching.
4	Explain <code>groupby()</code> with an example.	It splits data into groups, applies an aggregate function per group, and combines the results — e.g., <code>df.groupby('City')['Sales'].sum()</code> .
5	What is the difference between <code>apply()</code> and <code>map()</code> ?	<code>map()</code> works element-wise on a single Series (often for value lookups); <code>apply()</code> can operate on a Series or across rows/columns of a DataFrame for more complex logic.
6	How do you remove duplicate rows?	<code>df.drop_duplicates()</code> , optionally with <code>subset=</code> and <code>keep=</code> parameters.
7	What does the <code>inplace</code> parameter do?	When <code>True</code> , the operation modifies the DataFrame directly and returns <code>None</code> instead of returning a new object.
8	How do you convert a column to datetime?	<code>pd.to_datetime(df['col'])</code> , optionally with a format string.
9	What is method chaining in Pandas?	Calling multiple DataFrame methods in a single fluent expression, e.g. <code>df.dropna().sort_values('x').reset_index(drop=True)</code> .
10	What is the category dtype and why use it?	A dtype for columns with a limited number of repeated string values; it dramatically reduces memory usage and can speed up <code>groupby</code> operations.

14.3 Advanced (21–30)

#	Question	Answer
1	What is a MultiIndex and when does it appear?	A hierarchical index with multiple levels, commonly produced by <code>groupby()</code> on multiple columns or by <code>set_index()</code> with multiple columns.
2	Explain the difference between a view and a copy in Pandas.	A view shares memory with the original data (edits affect both); a copy is independent. Chained indexing often triggers the ambiguous 'SettingWithCopyWarning' because Pandas can't guarantee which one you have.
3	Why is vectorization faster than using <code>.apply()</code> with a lambda?	Vectorized operations run in compiled C code across an entire array at once; <code>.apply()</code> still loops row-by-row at the Python level, only with more convenient syntax.
4	How would you optimize memory usage of a large DataFrame?	Downcast numeric dtypes (<code>int64</code> -> <code>int32</code> , <code>float64</code> -> <code>float32</code>), convert low-cardinality text to category, drop unused columns, and read only needed columns via <code>usecols</code> .
5	What is the SettingWithCopyWarning and how do you avoid it?	It warns that an assignment may not persist because it was made on a view rather than the original DataFrame. Avoid chained indexing (<code>df[mask]['col'] = x</code>) and use <code>df.loc[mask, 'col'] = x</code> instead.
6	How do window functions like <code>rolling()</code> and <code>expanding()</code> work?	<code>rolling(n)</code> computes a statistic over a fixed-size sliding window (e.g., 7-day moving average); <code>expanding()</code> computes it over all data up to the current row (a cumulative window).
7	How would you merge two very large DataFrames efficiently?	Ensure join keys are indexed/sorted, use the correct dtype for keys, filter down to needed columns first, and consider chunked processing or Dask for data too large for memory.
8	What is the difference between <code>pivot()</code> and <code>pivot_table()</code> ?	<code>pivot()</code> reshapes data without aggregation and requires unique index/column combinations; <code>pivot_table()</code> supports aggregation (default mean) and can handle duplicate combinations.
9	What does the <code>query()</code> method offer over boolean indexing?	More readable, SQL-like syntax, and for very large DataFrames it can be faster because it uses the numexpr engine internally.
10	How do you profile and speed up a slow Pandas pipeline?	Use <code>%timeit</code> / <code>cProfile</code> to find bottlenecks, replace loops/apply with vectorized operations, optimize dtypes, and avoid unnecessary intermediate copies.

14.4 Scenario-Based Questions

These test applied judgement rather than textbook recall — exactly what senior interviewers probe for.

#	Question	Answer
1	You load a CSV and a numeric column shows as dtype 'object'. What might be wrong, and how do you fix it?	The column likely contains stray characters, commas, or mixed text ('N/A'). Inspect with <code>df['col'].unique()</code> , clean the values, then use <code>pd.to_numeric(df['col'], errors='coerce')</code> .
2	A merge produced far more rows than either input table. What happened?	Duplicate keys on one or both sides created a many-to-many match (a 'fan-out'). Check duplicates before merging and consider <code>validate='one_to_one'</code> in <code>merge()</code> to catch this early.
3	Your manager wants total revenue by month and region, sorted by revenue. How would you build this in one pipeline?	<code>df.groupby(['Region', 'Month'])['Revenue'].sum().reset_index().sort_values('Revenue', ascending=False)</code>
4	A dataset is too large to fit comfortably in memory. What are your options in the Pandas ecosystem?	Read in chunks with <code>chunksize</code> in <code>read_csv()</code> , downcast dtypes, drop unneeded columns early, or move to Dask/Polars which offer Pandas-like APIs for out-of-core or faster processing.
5	You need to flag customers whose latest order was more than 90 days ago. How would you	Group by customer to find the max order date, compute the difference from today with a <code>Timedelta</code> , and flag rows where that difference exceeds 90 days.

#	Question	Answer
	approach it?	

TRAINER TIP

In the final 15-minute Q&A slot, pick 5 random questions from this bank and have students answer verbally before showing the model answer — verbal recall under mild pressure closely simulates the real interview experience.

15. Best Practices

15.1 Writing Clean Pandas Code

- Chain methods thoughtfully with line breaks for readability instead of one giant unreadable line
- Use named aggregations in `agg()` instead of cryptic multi-index column names
- Prefer descriptive variable names (`monthly_revenue`, not `df2`)
- Reset the index after `groupby` operations when the result will be merged or exported

15.2 Avoiding Chained Indexing

```
# Risky - may trigger SettingWithCopyWarning, may silently fail
df[df['Age'] > 25]['Score'] = 0

# Correct
df.loc[df['Age'] > 25, 'Score'] = 0
```

15.3 Efficient Filtering

- Filter early — reduce the DataFrame to only needed rows/columns before heavy computation
- Combine multiple boolean conditions in a single filter pass instead of sequential re-filters
- Use `query()` for readability on complex multi-condition filters

15.4 Memory Optimization

- Convert repetitive text columns to category dtype
- Downcast numeric columns where the full precision isn't needed (`float64` -> `float32`)
- Load only required columns with `usecols` in `read_csv()`

15.5 Vectorization Over Loops

- Default to column-wise arithmetic and boolean masks
- Reach for `np.where()` / `np.select()` for conditional logic before `apply()`
- Reserve `apply()`/loops for logic that genuinely cannot be vectorized

15.6 Common Mistakes Recap

Mistake	Why It's a Problem	Better Approach
Using <code>and/or</code> on Series conditions	Raises <code>ValueError</code> on ambiguous truth value	Use <code>&</code> / <code> </code> with parentheses
Chained indexing for assignment	May silently fail or warn	Use <code>.loc[]</code> for reads and writes
Calling <code>apply()</code> for simple arithmetic	Slower than vectorized operations	Use direct column arithmetic

Mistake	Why It's a Problem	Better Approach
Ignoring index alignment on merges	Misaligned or duplicated rows	Verify keys and use <code>validate=</code> in <code>merge()</code>
Not checking dtypes after <code>read_csv()</code>	Silent type errors downstream	Always run <code>df.info()</code> first

15.7 Performance Tips Summary

- Vectorize wherever possible
- Use category dtype for repetitive text
- Avoid `iterrows()` and `itertuples()` unless absolutely necessary
- Filter and select columns before expensive operations, not after
- Profile before optimizing — measure, don't guess

16. Pandas Cheat Sheet

A dense, at-a-glance reference for the most-used Pandas operations. Ideal to print and keep next to your keyboard.

Series, DataFrames & File I/O

Task	Code
Create Series	<code>pd.Series([1,2,3])</code>
Create DataFrame	<code>pd.DataFrame({'a':[1,2]})</code>
Read CSV	<code>pd.read_csv('f.csv')</code>
Read Excel	<code>pd.read_excel('f.xlsx')</code>
Read JSON	<code>pd.read_json('f.json')</code>
Write CSV	<code>df.to_csv('f.csv', index=False)</code>
Write Excel	<code>df.to_excel('f.xlsx', index=False)</code>
First/last rows	<code>df.head()</code> / <code>df.tail()</code>
Structure info	<code>df.info()</code>
Summary stats	<code>df.describe()</code>
Shape	<code>df.shape</code>
Column names	<code>df.columns</code>
Data types	<code>df.dtypes</code>

Selecting, Filtering & Cleaning

Task	Code
Select column	<code>df['col']</code>
Select rows by label	<code>df.loc[0:2]</code>
Select rows by position	<code>df.iloc[0:2]</code>
Boolean filter	<code>df[df['col'] > 5]</code>
Multiple conditions	<code>df[(cond1) & (cond2)]</code>
isin filter	<code>df[df['col'].isin([...))]</code>
Range filter	<code>df[df['col'].between(a,b)]</code>
String filter	<code>df.query("col > 5")</code>
Missing values	<code>df.isnull().sum()</code>
Drop missing	<code>df.dropna()</code>
Fill missing	<code>df.fillna(value)</code>
Drop duplicates	<code>df.drop_duplicates()</code>
Rename columns	<code>df.rename(columns={'a':'b'})</code>

Manipulation, GroupBy, Merge & Dates

Task	Code
Sort values	<code>df.sort_values('col')</code>
Sort index	<code>df.sort_index()</code>
Apply function	<code>df['col'].apply(func)</code>
Map values	<code>df['col'].map(dict_or_func)</code>
GroupBy + aggregate	<code>df.groupby('col').agg(...)</code>
Multiple aggregations	<code>df.groupby('col')['x'].agg(['sum', 'mean'])</code>
Concat (stack)	<code>pd.concat([df1, df2])</code>
Merge (SQL join)	<code>pd.merge(df1, df2, on='key', how='left')</code>
Join on index	<code>df1.join(df2)</code>
To datetime	<code>pd.to_datetime(df['col'])</code>
Extract date part	<code>df['col'].dt.year</code>
String methods	<code>df['col'].str.lower()</code>
Value counts	<code>df['col'].value_counts()</code>

Useful Aggregation Functions

Function	Purpose
<code>sum()</code> , <code>mean()</code> , <code>median()</code>	Central tendency measures
<code>min()</code> , <code>max()</code>	Range boundaries
<code>std()</code> , <code>var()</code>	Spread / variability
<code>count()</code> , <code>size()</code>	Number of (non-null) records
<code>nunique()</code>	Count of distinct values
<code>first()</code> , <code>last()</code>	First/last value per group

17. Summary

Over the past three hours, we covered the complete Pandas toolkit needed for real-world data analytics:

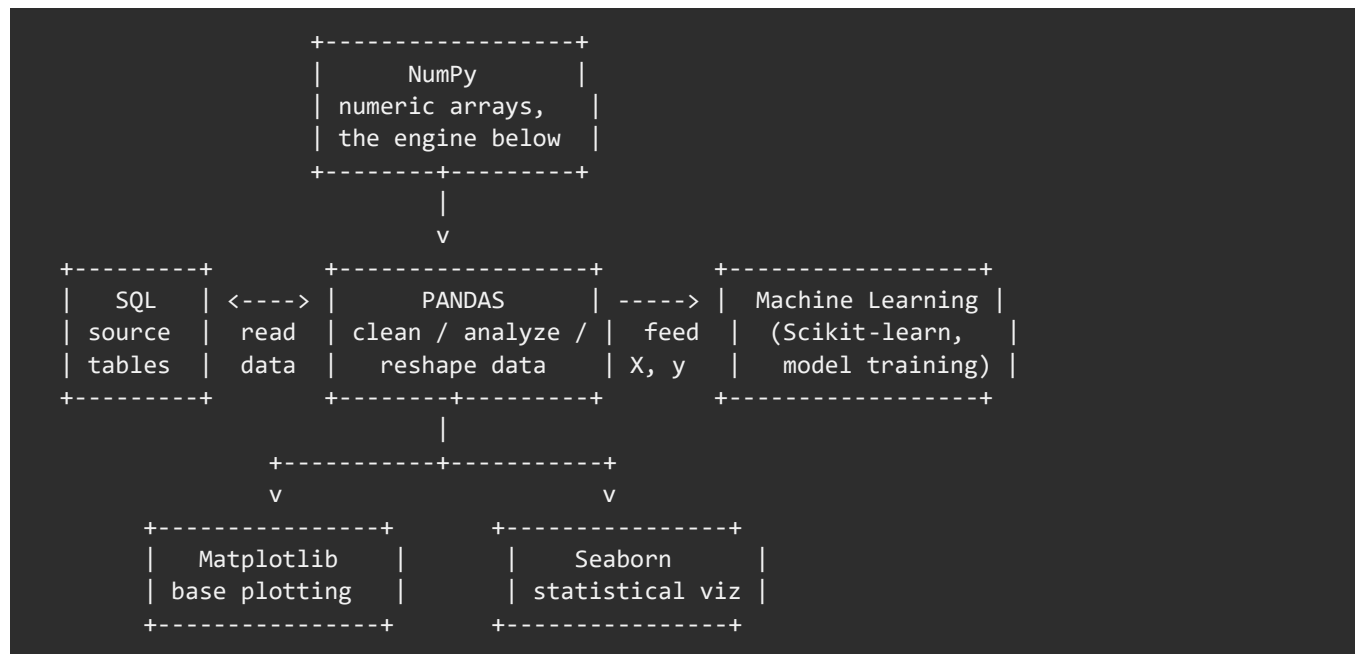
- Introduction — what Pandas is, why it beats raw Python and Excel, and how it relates to NumPy
- Data Structures — Series and DataFrames, how to create and inspect them
- Reading & Writing Files — CSV, Excel, JSON, and HTML I/O
- Exploring Data — head, info, describe, and value_counts as a standard first-look routine
- Selecting Data — loc, iloc, boolean filtering, isin, between, and query
- Data Cleaning — missing values, duplicates, string cleaning, and dtype fixes
- Data Manipulation — sorting, apply/map, and vectorized operations
- GroupBy & Aggregation — the split-apply-combine pattern for business reporting
- Merge, Join & Concatenate — combining datasets the way SQL does
- Date & Time — parsing, extracting, filtering, and arithmetic on dates
- Performance & Optimization — vectorization, memory tuning, and avoiding slow patterns
- A full hands-on project — the Student Performance Analytics System, start to finish
- Practice exercises, 30 interview questions, best practices, and a quick-reference cheat sheet

CLOSING THOUGHT

Pandas rewards consistent, hands-on practice far more than passive reading. The fastest way to retain everything covered today is to rebuild the Student Performance project from scratch, from memory, within the next 48 hours.

Pandas Learning Roadmap

Pandas is the connective hub of the Python data analytics ecosystem. Here is how it fits with the tools you'll want to learn next.



How These Pieces Connect

Tool	Role	Typical Handoff From Pandas
NumPy	Underlying numeric engine	Pandas is built on NumPy arrays
SQL	Source of structured data	<code>pd.read_sql()</code> pulls query results into a DataFrame
Matplotlib	Base plotting library	<code>df.plot()</code> is built directly on Matplotlib
Seaborn	Statistical, styled visualizations	Accepts a DataFrame directly, e.g. <code>sns.barplot(data=df)</code>
Scikit-learn	Machine learning models	Cleaned DataFrame columns become model features (X) and target (y)

Suggested Next Steps

- Week 1-2: Deepen Pandas — multi-indexing, `pivot_table()`, window functions (rolling/expanding)
- Week 3: Learn Matplotlib and Seaborn to visualize the DataFrames you've been building
- Week 4: Learn SQL fundamentals — the mental model (SELECT/GROUP BY/JOIN) maps almost directly onto Pandas
- Month 2: Introduce Scikit-learn — use cleaned Pandas DataFrames as direct input to your first ML models
- Ongoing: Rebuild real projects end-to-end (like today's Student Performance system) using public datasets from Kaggle

FINAL WORD

Every advanced tool in this ecosystem assumes you can already get data into a clean DataFrame. That is exactly the skill this workshop built today — the rest of the roadmap is now much easier to climb.

© Shalinee Priya